AI-037: MODULE SUBSYSTEM POWER ESTIMATOR DESIGN DOCUMENT

by

Kenn Du

Songli Du

Lucas Song

Riley Zhang

Sonya Zhao

April 9, 2026

Table of Contents

1. High-Level Description of Product **3**
1.1 Major components/subsystems 3
2. Design Components (integration, choices, reasoning, eliminated choices) **4**
2.1 Programming Language: Python 4
2.2 Software Architecture: Object-Oriented Programming (OOP) 5
2.3 Data Interchange Format: JSON 5
2.4 Power Calculation Methodology 6
3. Detailed Component Design **6**
3.1 Input Parser Module 6
3.2 Core Power Calculator 7
3.3 Interface Power Calculator 8
3.4 Component Power Calculator 9
3.5 DIMM Module 10
4. Backend API Module **10**
4.1 API Architecture 10
4.2 Request/Response Models 11
4.3 Integration with Core Package 11
4.4 Error Handling 12
4.5 Deployment Integration 12
5. Full Stack Module **13**
5.1 Frontend Architecture 13
5.2 Calculation Source and Role of the API 13
5.3 API Client Configuration and Request Shaping 13
5.4 Server Deployment Designer 14
5.5 Target Power (Inverse DDR5) 14
5.6 Visual DIMM Builder 14
5.7 Configuration Hub 15
6. Testing Strategy **15**
6.1 Unit Testing Strategy 15
6.2 Integration Testing Approach 15
6.3 E2E Testing 16
6.4 Test Coverage Goals 16
6.5 Continuous Integration 16
7. Team Contributions **17**

List of Figures

Figure 1. Block diagram of the project's current high-level design. 4
Figure 2. Core power calculation endpoint implementation. 11
Figure 3. Vercel serverless function entry point. 12
Figure 4. Backend module file structure. 12

1. High-Level Description of Product

The Use-Case Power Estimator is a software tool designed to estimate the power consumption of DDR5 and LPDDR5 memory subsystems. Unlike cycle-accurate simulators (e.g., DRAMSys) that simulate every clock edge, this tool employs a high-level, use-case driven methodology.. It calculates power by aggregating memory command statistics over a specific duration and applying JEDEC-standard current/voltage specifications.

1.1 Major components/subsystems

The system follows a modular Input-Process-Output (IPO) architecture, consisting of three main subsystems:

1. **Input Parser**: Ingests specifications (DRAM parameters) and workloads (Command counts) from JSON files.
2. **Power Calculator Engine**: The core logic that applies power equations based on the specific memory standard (DDR5, later LPDDR5).
3. **Visualization & Reporting Module:** Processes the calculated data to generate total power figures and plots.

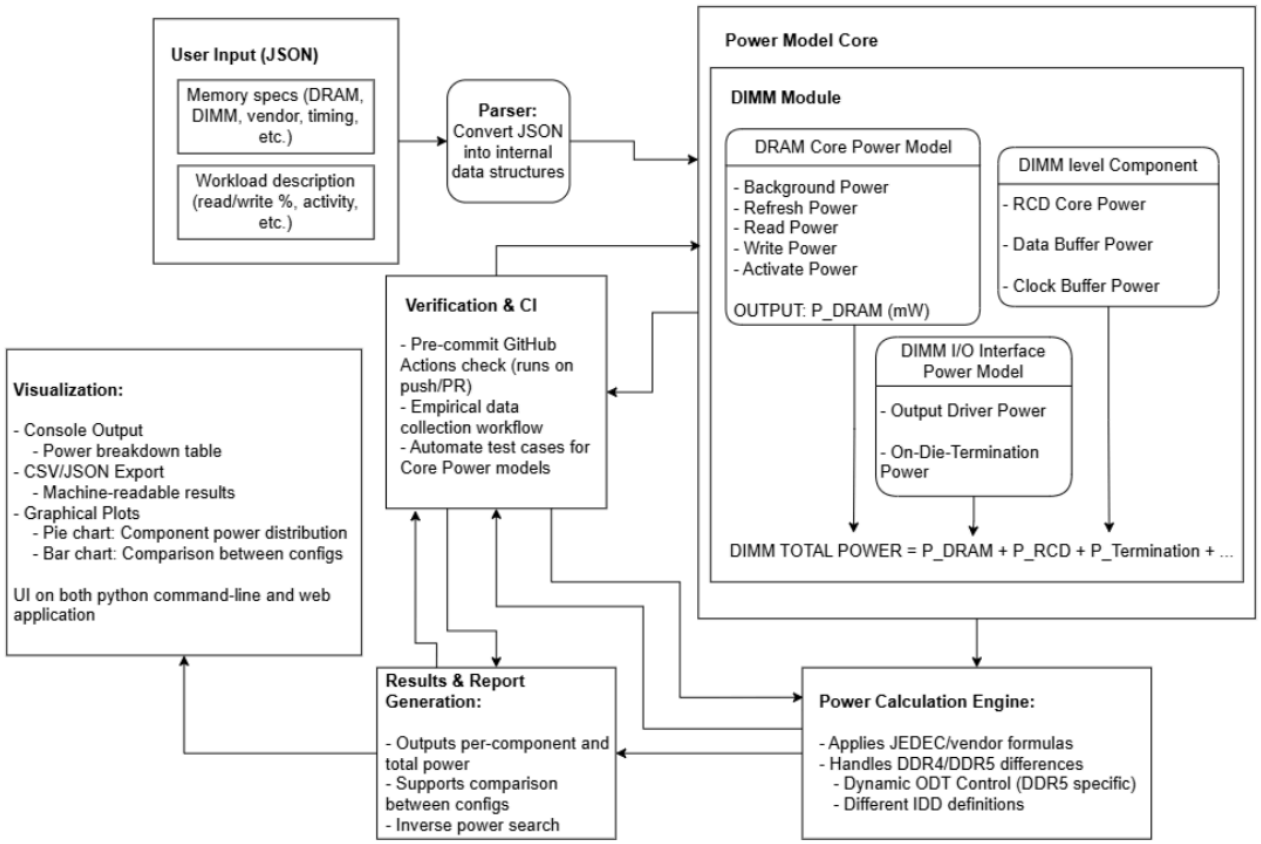

Figure 1. Block diagram of the project's current high-level design.

2. Design Components (integration, choices, reasoning, eliminated choices)

This section details the specific technological choices and the engineering reasoning behind them, linking back to the Requirements Document.

2.1 Programming Language: Python

Choice: The tool is implemented entirely in Python.

Rationale:

- **Visualization:** Python provides robust libraries (matplotlib, pandas) for generating the required charts (F-03, F-04).

- **Performance:** The tool implements a use-case–level power model, which relies on arithmetic operations rather than cycle-by-cycle simulation. This approach is not computationally intensive; therefore, the performance advantage C++ provides is not required to meet the performance requirement of processing workloads in an acceptable amount of time (NF-02).

Integration:

- **Data Processing:** pandas and numpy for efficient handling of large workload datasets.

Alternatives Eliminated:

- **C++**: While used by DRAMPower and DRAMSys, C++ was rejected. It introduces unnecessary complexity for JSON parsing and plotting, and its performance benefits are negligible for our purposes.

2.2 Software Architecture: Object-Oriented Programming (OOP)

Choice: The codebase is structured using class-based object-oriented programming to support extensibility (NF-03).

Rationale:

- **Extensibility**: The project intends to support both DDR5 and LPDDR5 in further development, also, support DIMMs and different components that aren't DRAM. These standards share common behaviours but differ in voltage rails and power states. A base class with inherited DDR5 and LPDDR5 subclasses allows code reuse while handling standard-specific logics. (F-01)
- **Component Modelling**: OOP allows us to model the "DIMM" as an object containing "Chip" objects and "PMIC" objects, accurately reflecting the physical hardware hierarchy.

Integration:

- **Classes**: These are organized into separate modules to enforce separation of concerns and simplify unit testing integration.

2.3 Data Interchange Format: JSON

Choice: Both inputs (DRAM specifications and Workload scenarios) are structured as JSON files and provide the required inputs as described by C-03.

Rationale:

- **Configurability**: JSON allows the tool to run different vendor specifications (e.g., Micron vs. Samsung) or different speeds (4800 vs. 6400 MT/s) without modifying the source code. (NF-03)
- **Interoperability:** The structure mimics the existing DRAMPower resource files, allowing us to validate our inputs against established open-source tools.

Integration:

- Natively supported by Python, requiring no external dependencies for parsing.

2.4 Power Calculation Methodology

Choice: Adaptation of Micron's DDR4 Power Equations for DDR5/LPDDR5.

Rationale:

- **Accuracy (NF-01)**: We utilize the standard summation method: E_total = ∑ (I_state * V_supply * t_state).
- **Adaptation**: We are adapting the proven equations used in industry (Micron's DDR4 models) but replacing the parameters with DDR5-specific values (e.g., replacing V_DD=1.2V with V_DD=1.1V and adding the V_PP rail).

Integration:

- **Equations:** Directly use data from the JSON specification files (e.g., IDD currents, VDD voltage values).

3. Detailed Component Design

3.1 Input Parser Module

Function: Reads DRAM specification and workload json files and converts them into dataclass usable by the calculator.

Design: Deserializes the vendor specification JSON into strict dataclass objects (MemSpec, MemPowerSpec, etc.) to ensure type safety. Field access is achieved via class hierarchy such as memspec.mempowerspec.idd4r. For workloads, the module converts user workload inputs (e.g.,

percentage-based activity) into Workload dataclass used in the calculator.

Error handling & reliability: validation and error handling to meet requirement C-03. Checks top-level sections (memarchitecturespec, mempowerspec, memtimingspec), and verifies all required power specification fields (21 IDD/IPP current values and voltage parameters) are present. Raises explicit exceptions when failures are detected: (e.g., "Missing required power spec field(s): vdd, idd2n") or invalid type conversions (e.g., "Invalid value in memtimingspec: could not convert string to float").

3.2 Core Power Calculator

Power estimation for a single DRAM device requires voltage and current. However, within a memory system, each DRAM chip may be in one of several different states at any given time. For example, in order to read a particular stored value, the DRAM row containing that stored value must first be “activated” and brought into the DRAM row buffer, which incurs a current cost (i). Then, power is required to keep that row active in the row buffer, which incurs another current cost (ii). Then to actually complete the read operation, additional energy is required to move data from the row buffer to the data pins connected to the host device, which incurs a third current cost (iii).

From these IDD specifications provided by vendors, the tool calculates power consumption under specific workloads as the product of the IDD current and operating voltage, scaled based on the percent utilization of any particular DRAM state (depending on workload). The total power estimation is then the sum of all the power contributions from these different states.

Background Power Example

The IDD currents of note for background power are IDD2N, IDD2P, IDD3N, IDD3P. These are respectively:

- IDD2N - Precharge standby: clock enabled, but no DRAM row has been opened yet for reading/writing (additional current needs to be spent to open a row before a read or write can occur).
- IDD2P - Precharge power-down: clock disabled, no DRAM row has been opened yet for reading/writing.

- IDD3N - Active standby: clock enabled, and at least one row has been opened for reading/writing.
- IDD3P - Active power-down: clock disabled, but at least one row is opened for reading/writing.

Percentage of time DRAM stays in power down state:

- LO_PRE_frac: Fraction of time the DRAM is in precharged power-down; corresponds to IDD2P
- LO_ACT_frac: Fraction of time the DRAM is in active power-down; corresponds to IDD3P

When all banks are precharged, the device alternates between precharge standby (IDD2N) and precharge power-down (IDD2P) according to the fraction of time power-down is enabled; similarly, when at least one bank is active, the device alternates between active standby (IDD3N) and active power-down (IDD3P). The corresponding average currents are converted to power using the VDD supply voltage, and the final background power is obtained by weighting the precharged and active background powers by the fraction of time the device spends in each bank state, BNK_PRE_frac in this case.

3.3 Interface Power Calculator

<u>Physics-Based Calculation (I^2R Model)</u>

The enhanced model replaces the simplified parallel formula with a series circuit analysis. In a real transmission line, the voltage source, driver impedance (R_ON), and termination resistance (R_TT) form a series loop. We calculate the specific loop current (I_loop) to determine the exact power dissipated at the DRAM die:

$$I_{loop} = \frac{V_{DDQ}}{R_{ON}+R_{TT}}$$

This allows for precise power attribution based on signal direction:

- During Writes (DRAM as Receiver): The DRAM consumes Termination Power.

$$P_{DRAM} = I_{loop}^2 \times R_{TT_DRAM}$$

- During Reads (DRAM as Driver): The DRAM consumes Driver Power (internal heat generation).

$$P_{DRAM} = I_{loop}^2 \times R_{ON_DRAM}$$

This correction eliminates the overestimation inherent in the baseline model by accounting for the voltage drop across the driver.

Note: These equations are team-derived from standard I^2R circuit relations using JEDEC-defined DDR5 termination, drive, and output characteristics. This formulation is informed by JEDEC JESD79-5, Section 3.5.1, and Micron DDR5 documentation.

<u>Integration of DDR5 Signal Topology</u>

The model is being expanded to support DDR5-exclusive signals that were generalized in the baseline:

- WCK (Write Clock): A high-frequency differential clock specific to DDR5, modeled strictly during Write cycles.
- CA ODT (Command/Address Termination): Unlike DDR4, DDR5 utilizes ODT on the Command Bus. The updated model calculates this based on $MR33$ register definitions.

3.4 Component Power Calculator

The final model includes power estimation for external components on the DIMM. Currently, the model calculates the power of a Registered Clock Driver (RCD) and data buffers (DBs) located on the DIMM device. RCDs are only present on Registered Dual-Inline Memory Modules (RDIMMs) which are commonly used in datacenters and servers. They are used to buffer the command and address buses between the CPU and DRAM to ease electrical load on the memory controller, increasing stability for high memory capacities. Data buffers are found on Load-Reduced DIMMs (LRDIMMs) which further increase stability by buffering the data lines as well.

The power of both these components are calculated using two key IDD values:

- IDD3NA - Active Idle Current: Component is active and idle, no commands are processed.
- IDD4A: Active Current: Component is actively driving commands/addresses/data lines.

These values made by component vendors are not publicly available, so for the purposes of our model, IDD3NA is represented by the previously mentioned IDD3N value and IDD4A is represented by IDD4W/IDD4R, depending on read or write workload.

3.5 DIMM Module

dimm.py defines the DIMM class. This class serves as the aggregation layer that combines the core, interface, and component-level power models. This class reads the workload and memory spec inputs and provides them to the other model classes, then calls their compute methods to calculate the total power used by the DIMM. The power models are now instance objects that the DIMM class can access.

4. Backend API Module

Function: Provides a RESTful API interface for the DDR5 power calculator, enabling programmatic access to power calculations from web clients and other services.
Design: The backend is implemented using FastAPI, a modern Python web framework that provides automatic OpenAPI documentation and type validation. The API serves as a bridge between the frontend web application and the core calculation engine, handling request validation, data transformation, and error handling.

4.1 API Architecture

The API follows a RESTful design pattern with the following endpoints:
GET / and GET /health: Health check endpoints for service monitoring
POST /api/calculate/core: Calculates core DRAM power consumption
POST /api/calculate/interface: Calculates interface/I/O power consumption
POST /api/calculate/all: Calculates both core and interface power in a single request

POST /api/calculate/dimm: Calculates DIMM-level power by aggregating multiple DRAM devices

4.2 Request/Response Models

The API uses Pydantic models for request validation and type safety. The main request model PowerCalculationRequest contains:

memspec: Memory specification with architecture, power, and timing parameters

workload: Workload characteristics including read/write percentages and timing constraints

Each endpoint returns a dictionary of power values in Watts, with keys such as P_total_core, P_VDD_core, P_VPP_core, P_total_interface, and component-level breakdowns.

```python
@app.post("/api/calculate/core", response_model=Dict[str, float])
async def calculate_core_power(request: PowerCalculationRequest):
    """
    Calculate DDR5 core power consumption.

    Returns a dictionary with power breakdown values in Watts.
    """
    try:
        memspec = memspec_model_to_obj(request.memspec)
        workload = workload_model_to_obj(request.workload)

        core_model = DDR5CorePowerModel()
        ddr5 = DDR5(memspec, workload, core_model=core_model)

        result = ddr5.compute_core()
        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

Figure 2. Core power calculation endpoint implementation.

4.3 Integration with Core Package

The backend imports the core calculation modules directly from the core/src directory. Request models are converted to core dataclass objects (MemSpec, Workload) using conversion functions (memspec_model_to_obj, workload_model_to_obj). The API instantiates DDR5CorePowerModel and DDR5InterfacePowerModel classes and orchestrates the calculation workflow through the DDR5 and DIMM wrapper classes.

4.4 Error Handling

The API uses FastAPI's exception handling to catch calculation errors and return appropriate HTTP status codes (500 for server errors, 422 for validation errors). Error messages are propagated to the client for debugging while maintaining security.

4.5 Deployment Integration

The backend includes a Vercel serverless function entry point (index.py) that wraps the FastAPI application using Mangum, enabling deployment on Vercel's serverless infrastructure. The function handler is compatible with AWS Lambda-style events used by Vercel.

```python
# Vercel serverless function entry point
import sys
from pathlib import Path

# Add core/src to path
project_root = Path(__file__).parent.parent.parent
core_src_path = project_root / "core" / "src"
sys.path.insert(0, str(core_src_path))

from main import app
from mangum import Mangum

# Wrap FastAPI app with Mangum for AWS Lambda/Vercel compatibility
handler = Mangum(app, lifespan="off")
```

Figure 3. Vercel serverless function entry point.

```
backend/
├── src/
│   ├── main.py          # FastAPI application and endpoints
│   └── index.py         # Vercel serverless function entry point
├── tests/
│   ├── conftest.py      # Pytest fixtures
│   ├── test_api_core.py # Core power API tests
│   └── test_api_interface.py # Interface power API tests
├── requirements.txt     # Python dependencies
├── vercel.json         # Vercel deployment configuration
└── README.md           # Backend documentation
```

Figure 4. Backend module file structure.

5. Full Stack Module

The web application provides configuration, power estimation, and visualization for DDR5 and LPDDR5/X. The Python core behind FastAPI is the reference implementation; the browser may use a TypeScript DDR5 path only when the API is absent or fails (§5.4). The UI is implemented with Next.js (App Router), React, TypeScript, and Tailwind CSS.

5.1 Frontend Architecture

Primary routes: / (landing), /configuration (inputs), /core-power, /dimm-power, /target-power (inverse search), /server-deployment (fleet-style DIMM selection), /visual-builder (DDR5 topology builder).
Shared MemSpec / Workload types align the UI with the parser and API. Client-side state uses React context and local state where interactivity is required.

5.2 Calculation Source and Role of the API

Authoritative behavior for deployment and regression is defined by the Python package. FastAPI exposes POST /api/calculate/* so the web app, CLI, and tests share one model—especially for LPDDR5/X, which is not silently approximated in the browser.

Normal use: the client calls the API (same-origin /api/... or an explicit base URL). DDR5-only exception: if the API is unavailable or errors, the UI may compute with computePowerLocal; that path is a resilience feature, not the regression baseline.

5.3 API Client Configuration and Request Shaping

Base URL: non-empty NEXT_PUBLIC_API_URL selects the API host (e.g. local backend or separate deploy). If unset in the browser, window.location.origin is used so calls hit /api/... on the same deployment. Empty string disables the API (tests, forced local-only).

normalizeMemspecForApi: supplies required API fields (e.g. registered as "true"/"false", defaults for nbrOfDevices / nbrOfDBs); for DDR5, converts UI currents from mA to A when values appear to be in milliamps. Responses are mapped via coreApiResponseToPowerResult and dimmApiResponseToResult, including LPDDR multi-rail fields when present.

5.4 Server Deployment Designer

Users specify power budget (per server or total fleet—with fleet mode using a fixed reference server count only to derive an equivalent per-server cap during matching), minimum MT/s, minimum capacity per server (GB), max DIMM slots, workload profile, and DIMM search mode (sweep 1…N slots vs N only). The tool scores every memory preset at the chosen workload using /api/calculate/dimm/batch when possible, else the JS calculator; builds valid (preset × DIMM count) configs; ranks by power headroom, performance margin, and mild overcapacity penalty; returns a short, diverse shortlist. Selection drives charts, fleet/rack-style summaries, and “Use configuration” → /configuration with that memspec and scenario workload.

5.5 Target Power (Inverse DDR5)

Users set a target core power (W) and optionally target DIMM total (W) on the current Configuration workload. The search perturbs each preset’s currents and VDD/VPP within bounds, evaluates candidates (batch API preferred, JS fallback on API error), and minimizes a weighted squared-error loss. Seeded PRNG from serialized inputs makes results reproducible. “Use in calculator” stores the winning memspec and opens /configuration. Scope: DDR5-style inverse flow; not a substitute for full LPDDR validation elsewhere.

5.6 Visual DIMM Builder

Users edit a logical DDR5 DIMM (ranks, bank groups, banks, x8/x16, columns, device count, speed tier) with a 3D board; architecture is derived into memarchitecturespec. Live power uses tryApiThenLocalPower. Available standalone at /visual-builder or embedded under Configuration → Build.

5.7 Configuration Hub

In-app: memory via presets or visual builder; workload via presets / ConfigPanel. Upload: validated memspec and workload JSON. Serves as the handoff point for core and DIMM views and for results pushed from §5.5 and §5.6.

6. Testing Strategy

Function: Ensures correctness, reliability, and maintainability of the power calculator across all system components.

Design: The testing strategy follows a multi-layered approach, covering unit tests, integration tests, end-to-end tests, and regression testing to validate both individual components and system-wide behavior.

6.1 Unit Testing Strategy

Unit tests focus on individual modules and classes in isolation:

Core Package Tests (core/tests/): Test DDR5CorePowerModel, DDR5InterfacePowerModel, parser functions, and dataclass validation. Tests validate power calculation correctness with known inputs and expected outputs.

Backend API Tests (backend/tests/): Test FastAPI endpoints using TestClient, validate request/response models, and verify error handling for invalid inputs.

Frontend Tests (frontend/tests/): Test React components, hooks, and utility functions using Jest and React Testing Library.

Unit tests use pytest for Python code and Jest for TypeScript/React code. Fixtures provide reusable test data (sample memspecs, workloads) to reduce duplication.

6.2 Integration Testing Approach

Integration tests validate interactions between components:

Backend Integration (backend/tests/test_api_core.py, test_api_interface.py): Test that API endpoints correctly invoke core calculation modules and return properly formatted results.

Core Package Integration: Verify that DDR5 and DIMM classes correctly orchestrate power models and aggregate results.

Integration tests use the same pytest framework but focus on component boundaries rather than isolated units.

6.3 E2E Testing

End-to-end tests (tests/e2e/) validate the complete system workflow:

Full Stack Integration (test_full_stack.py): Tests the entire flow from core package → backend API → response, ensuring that API results match direct core package calculations within acceptable floating-point precision (0.1% tolerance).

API Integration (test_api_integration.py): Verifies that the backend API correctly uses the core package and handles edge cases.

E2E tests use FastAPI's TestClient to simulate HTTP requests and validate end-to-end behavior without requiring a running server.

6.4 Test Coverage Goals

Core calculation logic: >90% coverage.

API endpoints: >80% coverage.

Critical error paths: 100% coverage.

Frontend components: Focus on user-facing functionality.

6.5 Continuous Integration

Tests are designed to run in CI/CD pipelines. The test suite can be executed with:

`pytest core/tests/` for core unit tests.

`pytest backend/tests/` for API tests.

`pytest tests/e2e/` for end-to-end tests.

`python core/test_regression.py` for regression validation.

7. Team Contributions

Section	**Major Contribution**	**Minor Contribution**	**Author**	**Reviewer**
1	Sonya, Riley, Songli	Lucas	Sonya	Lucas
2	Sonya, Riley	Lucas, Songli	Riley	Lucas
3	Sonya, Lucas	Riley, Songli	Sonya	Riley
4	Lucas		Lucas	Riley
5	Lucas		Lucas	Riley
6	Lucas		Lucas	Riley
7	Sonya	Lucas	Sonya	Lucas